

SketchVLM: Vision language models can annotate images to explain thoughts and guide users

Brandon Collins^{1*}
blc0063@auburn.edu

Logan Bolton^{1*}
logan.bolton@auburn.edu

Hung Huy Nguyen^{1*}
huyhung.dknec@gmail.com

Mohammad Reza Taesiri
mtaesiri@gmail.com

Trung Bui²
bui@adobe.com

Anh Totti Nguyen¹
anh.ng8@gmail.com

¹Auburn University

²Adobe Research

Abstract. When answering questions about images, humans naturally point, label, and draw to explain their reasoning. In contrast, modern vision-language models (VLMs) such as Gemini-3-Pro and GPT-5 only respond with text, which can be difficult for users to verify. We present SketchVLM, a training-free, model-agnostic framework that enables VLMs to produce non-destructive, editable SVG overlays on the input image to visually explain their answers. Across seven benchmarks spanning visual reasoning (maze navigation, ball-drop trajectory prediction, and object counting) and drawing (part labeling, connecting-the-dots, and drawing shapes around objects), SketchVLM improves visual reasoning task accuracy by up to +28.5 percentage points and annotation quality by up to $1.48\times$ relative to image-editing and fine-tuned sketching baselines, while also producing annotations that are more faithful to the model’s stated answer. We find that single-turn generation already achieves strong accuracy and annotation quality, and multi-turn generation opens up further opportunities for human-AI collaboration. An interactive demo and code are at <https://sketchvlm.github.io/>.

Keywords: Vision-language models · Explainable AI · Computer Vision

1 Introduction

From browsers (Atlas, MS Edge [39]) to office products (MS Copilot, Gemini [43]), the market for LLM-powered chatbots for general end users is estimated to grow from \$1.1B in 2023 to \$83B in 2032 [6]. As these assistants increasingly answer image-based questions in high-frequency consumer and productivity workflows [6, 35, 39, 43], users need responses they can quickly understand and verify [29].

However, modern VLMs such as Gemini-3-Pro and GPT-5 typically respond with a block of text, which can be difficult to verify [55] (Fig. A2). To explain

* All three co-first authors made major contributions to dataset creation, experiments, and manuscript. See Sec. 7 for a detailed author contribution statement.

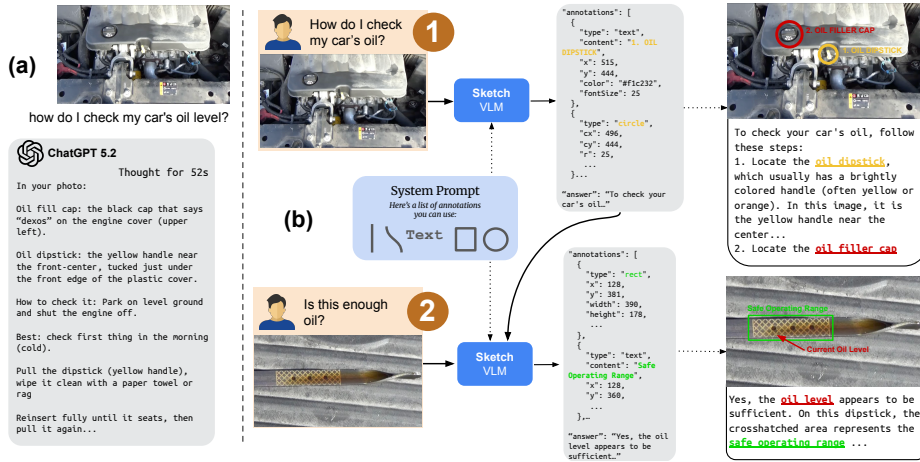


Fig. 1: For complex questions, modern chatbots like ChatGPT often return long text responses (a) that are hard for users to understand, verify, and follow. In contrast, SketchVLM guides users (b) step-by-step by annotating the input image and grounding answers to relevant image regions—here, guiding a user how to check their car’s oil level (source: <https://www.youtube.com/watch?v=tNNyu9S65E4>).

their thoughts on an image problem, humans often point, circle, underline, label, and annotate directly on the image. For example, when a user asks to check a car’s oil level, a visual annotation is easier to verify than a paragraph of text (Fig. 1) [26, 31].

Some VLMs emit point coordinates for referencing objects (e.g., MoonDream [45], Molmo [2]), but they do not support free-form visual annotation. Other models are trained to generate visual reasoning traces but do not generalize well beyond their training domains [18, 48]. Image-editing models can visualize intermediate reasoning steps for multimodal questions [57], but risk altering the source image in unintended ways, undermining user trust [41].

To address these issues, we propose SketchVLM, a state-of-the-art (SotA) system that draws SVG annotations in a separate layer overlaid on top of the input image to explain its reasoning. SketchVLM grounds its annotations directly on the original image without modifying it and without requiring task-specific training. We test SketchVLM with multiple VLM backbones, including Gemini-3-Pro-Preview (✦) [36] and GPT-5 (🌀) [30]. We collectively refer to these models harnessed with SketchVLM as SketchVLMs and individually as ✦ sketchVLM and 🌀 sketchVLM. We compare against the leading alternative approaches for generating visual annotations such as the SotA image-editing model Nano Banana Pro (🍌) [13] and specialized VLMs fine-tuned to produce image annotations, ViLaSR (👁️) [48] and ThinkMorph (🧠) [18].

On a comprehensive evaluation of **seven tasks**: (a) **three drawing** tasks (connecting-the-dots, labeling parts of an object, and drawing shapes around

objects) and (b) **four visual reasoning tasks** (two physics understanding tasks, one counting task and one maze navigation task), our main findings are:

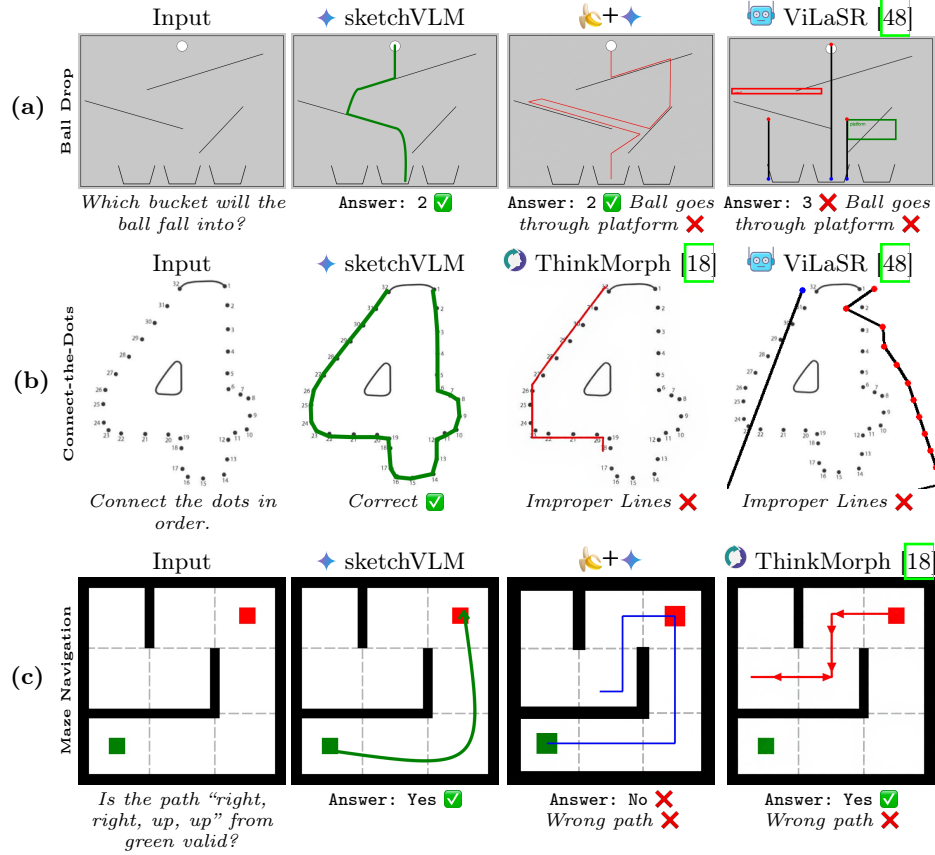


Fig. 2: Our 🔹 sketchVLM (Gemini-3-Pro-Preview) draws more accurate predicted trajectories in Ball Drop (a); connects the dots more accurately (b); and sketches more plausible maze navigation paths (c). Nano Banana (🍌+🔹) often undesirably alters the image and draws implausible trajectories in ball drop and maze navigation. Specialist VLMs (🧠 and 🤖) fine-tuned to sketch often fail to generalize to new tasks.

- SketchVLMs based on frontier models (🧠 and 🔹) generate annotations of superior **generalizability, accuracy, and annotation quality** compared to those by specialized fine-tuned sketching models (🤖, 🧠) (Secs. 5.2, 5.3 and 5.6 to 5.10).
- Nano Banana (🍌) is unable to generate a separate overlay layer and frequently alters the original image when generating in-image annotations (Secs. 5.4 and 5.5).

- SketchVLMs have similar accuracy in single-turn and multi-turn settings, but are significantly faster with single-turn generation (Sec. 5.11).
- Adding an external grid of x - y coordinates to the input image improves sketchVLM’s drawing capability and question answering accuracy but is not necessary for $\blacklozenge$ sketchVLM (Sec. 5.1).

2 Related Work

Native image-editing models (*e.g.*, GPT-Image-1.5 and 🦄) can directly modify images to add annotations, but their performance can be inconsistent for reasoning-heavy tasks [57]. Open-source native multimodal autoregressive models (*e.g.*, Chameleon [42] and Bagel [15]) enable interleaved text-image generation, but they lack an editable annotation layer aligned to the input. In contrast, SketchVLMs generate a non-destructive SVG overlay on the input image (Tab. 1).

Tool-calling and code generation Agentic systems improve visual reasoning by writing code that invokes external tools or manipulates the input image. V* [49] uses LLM-guided visual search to localize target objects in high-resolution images, then crops relevant regions for closer inspection. Visual Sketchpad [19] and OpenThinkIMG [40] equip VLMs with modular vision tools such as segmentation, object detection, and OCR to support multi-step reasoning. PyVision [54] generates Python code to draw structured overlays on input images across multiple turns. Other training-free methods leverage internal attention or gradient maps to automatically crop and zoom into salient regions [53]. These systems excel at fine-grained visual understanding, but they typically rely on external tools or code execution rather than prompting a single VLM to produce user-facing free-form annotations directly on the image.

Visual prompting *e.g.*, drawing coordinate points or horizontal lines directly on an input image can improve the vision capabilities of VLMs [20, 22, 50]. Similarly, SketchAgent [46] appends a coordinate grid to the edge of the input image to allow the model to reference precise x - y positions in the image.

Visual sketching Whiteboard-of-Thought [27] prompts an LLM to produce Matplotlib code rendered on a blank canvas to give the model a space to draw before responding to text-based questions. D2R [33] interleaves textual chain-of-thought with rendered visual drafts of its proposed actions overlaid on the input image at each reasoning step, enhancing dynamic spatial reasoning across multiple turns. VDLM [47] converts images into SVG and then into a more LLM-interpretable format to improve visual understanding. SketchAgent [46] and SketchFormer [38] focus on sketch generation as a standalone task on a blank canvas. In contrast, SketchVLMs generate non-destructive, editable SVG annotations directly on existing input images so that users can inspect the model’s reasoning without altering the source image.

Fine-tuned sketching models MVoT [23] fine-tunes Chameleon [42] to generate interleaved text and image reasoning traces, visualizing intermediate states to support multi-step spatial reasoning. LatentSketchpad [52] and DeepSketcher [51] both move visual reasoning into learned latent or embedding spaces.

LatentSketchpad is built with Gemma3 and Qwen2.5-VL-7B, while DeepSketcher uses Qwen2.5-VL-7B. ViLaSR [48] post-trains Qwen-2.5-VL-7B to sketch on the input image with an SVG overlay before responding. Similarly, ThinkMorph is fine-tuned from BAGEL-7B-MoT [15] to generate visual sketches that support its answers, though it directly edits the input image rather than overlaying SVG annotations. Unlike these approaches, we build a harness around SotA VLMs to enable them to annotate on input images by generating a layer of SVGs. Therefore, different from the literature, our approach is training-free and enjoys the generalizability of SotA VLMs to new domains.

Table 1: Comparison of sketching models and methods. *Annotation type* describes the visual artifact used during reasoning: *Vector overlay* denotes structured, non-destructive marks (e.g., strokes/boxes/text) aligned to an image or canvas, while *Image edit* denotes pixel-space image modification or synthesis that may change image content. *Input image* is marked only when a provided image is the primary visual context (vs. blank canvas or purely generative visual thoughts). *Free-form drawing* indicates support for arbitrary stroke-like annotations beyond a fixed mark set.

Model name	Training-free	Multi-turn	Input image	Free-form drawing	Annotation type
SketchVLM (Ours)	✓	✓	✓	✓	Vector overlay
VisualSketchPad [19]	✓	✓	✓	✗	Vector overlay
PyVision [54]	✓	✓	✓	✗	Vector overlay
SketchAgent [46]	✓	✓	✗	✓	Vector overlay
D2R [33]	✓	✓	✓	✗	Image edit
Whiteboard-of-Thought [27]	✓	✗	✗	✗	Vector overlay
ViLaSR [48]	✗	✓	✓	✗	Vector overlay
OpenThinkIMG [40]	✗	✓	✓	✗	Image edit
ThinkMorph [18]	✗	✗	✓	✗	Image edit
MVoT [23]	✗	✓	✓	✗	Image edit
LatentSketchpad [52]	✗	✓	✓	✗	Image edit
DeepSketcher [51]	✗	✓	✓	✗	Image edit

3 SketchVLM

SketchVLM combines three components: visual prompting to aid spatial reference, a system prompt that elicits structured stroke outputs, and XML-to-SVG conversion that renders those strokes as an overlay on the source image (Fig. 1).

Visual prompting To make VLMs draw more reliably on tasks that require precision, such as Connect-the-Dots, we follow SketchAgent [46] and append a coordinate grid to the left and bottom of each image, scaled to the image resolution (Fig. D12).

Input prompt To enable VLMs to generate annotations, we introduce a system prompt (Sec. F.2) that instructs the model to produce stroke sequences in a specific format (e.g., XML-style <s1>, <s2>, ... <sN> tags each containing a list of points) corresponding to reasoning steps. We provide instructions for drawing primitives including rectangles, arrows, text labels, straight lines, and Bézier curves. The model is then given the task prompt (e.g., “Which bucket will

the ball fall into?" (Fig. 2) and is instructed to annotate its reasoning on the image before responding with a final answer (Fig. 1).

SVG conversion We parse the model’s XML output to a standardized SVG output that can be overlaid on top of the original input image. If there are exactly two points in a stroke, we overlay a straight line. Otherwise, given a stroke described by m ordered samples $S_i = \{(x_j, y_j)\}_{j=1}^m$ and corresponding normalized timestamps $T_i = \{t_j\}_{j=1}^m$ with $t_j \in [0, 1]$, we fit a smooth Bézier curve and render it as SVG. Following SketchAgent [46], a least squares solution is found for the control points in a cubic Bézier curve. See Fig. F5 for an example of the stroke output and the corresponding overlay. Following SketchAgent, we run all experiments using XML format. Given that VLMs produce JSON at comparable quality and JSON is more human-readable, we use JSON for our interactive demo.

4 Evaluation

4.1 7 Tasks

1. Connect-the-Dots  contains 100 images spanning three subsets: 21 randomly generated dot patterns, 30 connect-the-dots puzzles derived from silhouette SVGs, and 49 worksheet-style images collected from online sources. Models must locate each dot and connect them in order (Secs. B.1 and D.3 and Fig. 2).

2. Counting Objects  contains 746 images drawn from CountBench [5, 34], and Pixmo-Count [14]. We include object counts from 0 to 10 and filter out unsuitable Pixmo-Count examples. Models must count the target objects and place numbered markers on each one (Secs. B.2 and D.4 and Fig. 5).

3. Drawing Shapes around Objects  uses 1,000 images selected from the 5,000-image COCO validation set [25]. We choose images to balance object count and object size across classes. Models must localize objects by drawing rectangles or ovals (Secs. B.3 and D.5 and Fig. 6).

4. Part Labeling  contains 985 images selected from PACO [37] and Pascal-Part [10], covering 52 object classes. We keep images with a single target object occupying at least 10% of the image area and with at least four annotated part labels, while maintaining class balance. Models must place the correct text labels at the corresponding part locations (Secs. B.4 and D.6 and Fig. 7).

5. Maze Navigation  contains 200 generated 3×3 grid mazes. We vary the shortest path length from 3 to 8 steps and create invalid paths by perturbing one direction in the ground-truth path. Models must trace a proposed path and determine whether it reaches the goal without crossing walls (Secs. B.5 and D.2 and Fig. 2).

6. VPCT is the Visual Physics Comprehension Test [8], which contains 100 hand-crafted images. Models must predict which container a dropped ball will land in (Sec. B.6 and Figs. 2 and D1).

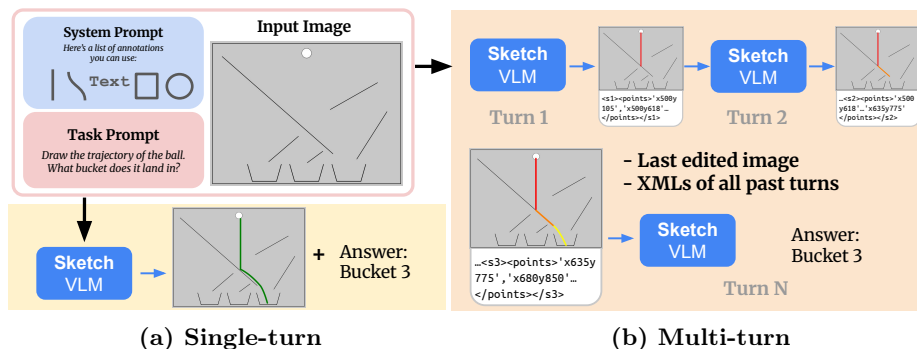


Fig. 3: Single-turn and multi-turn generation on the same VPCT sample. In (a) *single-turn*, SketchVLM receives the system prompt, the task prompt, and the input image, then outputs all annotations and the final answer in a single model call. In (b) *multi-turn*, Turn 1 uses the same inputs and outputs one annotation. For later turns, the model reuses the system prompt, the task prompt, and the previous annotations, which are provided in both the rendered image and text form. This process repeats until the model outputs its final text answer on the last turn.

7. Ball Drop  contains 198 synthetically generated images with harder-to-guess answers and ground-truth ball trajectories produced using PHYRE [4]. We generate equal numbers of images with 1, 2, and 3 randomly placed lines, and randomize the ball’s horizontal position. Models must predict the landing container out of four choices and trace the ball trajectory (Sec. B.6 and Figs. D2, D3 and 10).

4.2 Setup

Single-turn and multi-turn We evaluate SketchVLMs in (1) *single-turn*, where the models output all annotations and their final answer in one response, and (2) *multi-turn*, producing one stroke per turn, to simulate iterative, real-world conversations. During each turn, the VLM receives the image with all previous annotations rendered, and the annotations’ text representations. The model then outputs its final text answer on the last turn (Fig. 3).

Visual prompting We evaluate the necessity of a coordinate grid for the model to reference specific points in an image. When omitting the grid, the model outputs coordinates on a normalized 1000×1000 scale (Fig. D14).

4.3 Baselines

We compare SketchVLMs against three baselines that represent the SotA approaches for producing visual annotations on images (Fig. 4):

1. **Image Editing Model + VLM:**  is an image-editing model that can annotate images, but produces no text answer. To obtain a text response, we feed its edited image to  (denoted  + ).

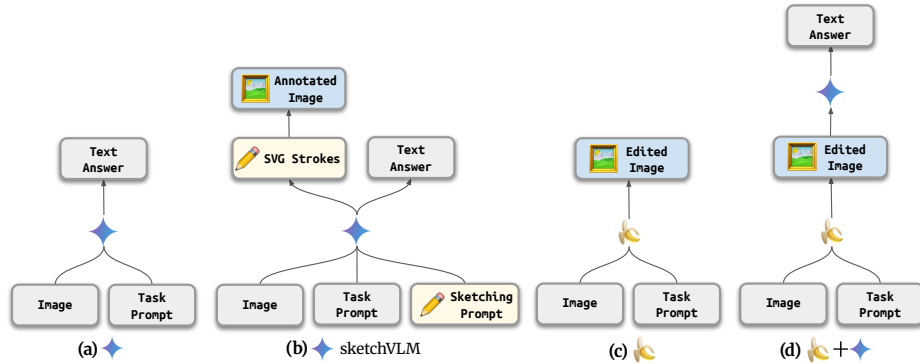


Fig. 4: Four approaches for making VLMs answer visual questions *and* annotate images.

- (a) $\blacklozenge$ outputs text only. No drawings generated.
- (b) $\blacklozenge$ sketchVLM draws on the image while outputting text.
- (c) ✂ only edits the image.
- (d) $\text{✂} + \blacklozenge$ takes the edited image from ✂ and gives it to $\blacklozenge$ to respond.

2. **Fine-tuned Sketching Models:** 🤖 is a model fine-tuned from Qwen-2.5-VL-7B [3] to autoregressively generate SVG annotations on the input image over multiple turns, and 🎨 is a model fine-tuned from BAGEL-7B-MoT [15] to directly edit the input image while also producing a text answer (Tab. F1 and Sec. G.1).
3. **Default VLMs:** We include text-only $\blacklozenge$ and 🗨 as simple baselines.

4.4 Metrics

A correct text answer from the model is not enough if the annotation is uninformative, and a plausible annotation can be misleading if it contradicts the text response. We therefore measure three distinct aspects of model performance:

Accuracy serves as the primary measure of task performance and tests the effect of sketching on answer correctness.

Annotation-text alignment measures how faithful the visual traces are to the text answer. We ask a VLM judge [9] to infer the answer from the annotations alone and report how often the judge’s inferred answer matches the model’s final text answer.

Annotation quality allows us to distinguish models that produce informative annotations from those that output low-quality or incoherent drawings. We adopt a VLM-as-a-Judge approach using a rubric scored from 1 to 5 (Sec. E) that evaluates annotation plausibility and visual clarity for each task.

Table 2: SketchVLMs produce visual reasoning traces while maintaining competitive accuracy. 🧑+🔹 underperforms default 🔹, and fine-tuned sketching models (🧑🔹, 🧑🔸) perform near random chance on visual reasoning tasks.

Model	VPCT					 draw shapes		 connect dots	
	video physics	ball drop	maze trace	counting	labeling			RMSE	Order Acc%
🔹 sketchVLM	96.0 ± 1.4	79.7 ± 2.8	98.0 ± 1.7	94.5	60.3	58.8	55.4	5.92	99.0
🔹	89.3 ± 2.2	83.8 ± 3.4	99.3 ± 0.8	93.0	64.1	63.1	59.8	†	†
🧑🔹 sketchVLM	70.0 ± 2.9	68.5 ± 2.2	92.8 ± 2.5	75.4	20.4	18.7	11.2	46.69	74.0
🧑🔹	63.5 ± 2.5	66.0 ± 4.7	92.3 ± 2.3	72.0	19.1	22.4	15.4	†	†
🧑+🔹	63.0	62.6	93.3 ± 3.0	91.7	†	†	†	†	39.0
🧑🔸	37.0	35.9	50.8 ± 1.5	48.6	—	—	†	198.74	9.0
🧑🔸	27.0	30.3	62.5 ± 2.1	68.1	†	†	†	†	0.0

† Model is unable to output this format.

5 Results

5.1 Grid prompting improves 🧑🔹 sketchVLM annotation precision, but hurts 🔹 sketchVLM

To understand how each component of our framework affects the model, we run ablations to see how sketching and grid prompting affect 🧑🔹 sketchVLM and 🔹 sketchVLM.

Experiment We evaluate four input configurations in single-turn mode: the base image alone, image with grid, image with sketching prompt, and image with both. We report accuracy on VPCT, Ball Drop, and Maze Navigation, and RMSE on Connect-the-Dots (Tab. 3).

Results 🧑🔹 performs best with both the grid and sketching prompt, with the grid providing a consistent boost to spatial precision (Fig. D12), consistent with prior findings [20]. 🔹 performs better without the grid, and adding it notably degrades localization on Connect-the-Dots (RMSE increases from 5.92 to 99.34). We therefore report all SketchVLM results in Tab. 2 with the input grid for 🧑🔹 but without the grid for 🔹 (Fig. D14).

5.2 SketchVLMs can localize points and connect them in order

Connecting the dots in an image tests both spatial grounding and the ability to produce multiple coherent strokes in a row.

Experiment We evaluate the root mean squared error (RMSE) in pixels of the difference between the ground truth position of each of the points and the SketchVLMs’ closest generated point. To evaluate whether models connect points in the correct order, we compare each predicted segment i against all ground-truth segment pairs using MSE. If segment i has lower MSE to a different ground-truth pair than its expected pair $(i, i+1)$, it is counted as an ordering error. Because 🧑+🔹 and 🧑🔸 only produce an image response with no x-y coordinates, we manually evaluate their outputs.

Results 🔹 sketchVLM can accurately output the correct location of up to 35 points with a very low RMSE of only 5.92 (Tab. 2). 🧑🔹 sketchVLM has a

Table 3: Ablation across inputs in single-turn mode. “Sketch” adds strokes/system prompt; “Grid” additionally overlays the coordinate grid. RMSE is reported for Connect-the-Dots while accuracy is reported for the other tasks. 🟩 sketchVLM works best with the grid while 🟦 sketchVLM works best without the grid

Model	Input	VPCT	🟩	🟦	✍️
🟩	Image	63.5	66.0	92.3	N/A
	+ Grid	67.0	67.2	98.0	N/A
	+ Sketch	59.0	63.1	88.7	134.40
	+ Sketch + Grid + Multi-turn	68.0	63.1	82.1	50.68
	+ Sketch + Grid (<i>i.e.</i> , 🟩 sketchVLM)	70.0	68.5	92.8	46.69
🟦	Image	89.3	83.8	99.3	N/A
	+ Grid	90.0	81.3	99.5	N/A
	+ Sketch (<i>i.e.</i> , 🟦 sketchVLM)	96.0	79.7	98.0	5.92
	+ Sketch + Multi-turn	80.0	79.8	98.2	12.37
	+ Sketch + Grid	91.0	82.3	99.5	99.34

higher RMSE of 46.69, but still performs much more reliably than 🟩’s 198.74 RMSE. Regarding ordering, 🟩, 🟦, and 🟩 frequently produce ordering errors as the number of strokes increases, often connecting points out of order and inadvertently altering the input image (Sec. C.2). In contrast, 🟩 sketchVLM and 🟦 sketchVLM correctly order points 74% and 99% of the time, respectively, demonstrating that SketchVLMs can reliably scale to many strokes while maintaining spatial accuracy and logical coherence (Fig. D13).

5.3 SketchVLM improves counting accuracy 🟦

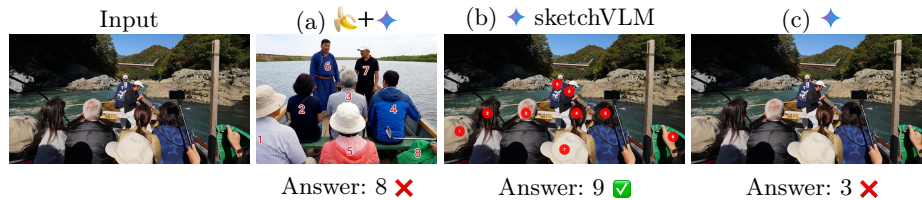


Fig. 5: (a) 🟩 + 🟦 generates a different image and predicts an incorrect count. (c) 🟦 directly outputs only a number without annotations and severely undercounts. In contrast, our 🟦 sketchVLM (b) outputs the correct answer and produces visual annotations to explain its answer.

Existing VLMs can output point coordinates to mark counted objects, but these points are unlabeled and can be tedious to verify. In contrast, SketchVLMs explicitly place numeric markers on each object, enabling direct visual verification of the predicted count.

Experiment We measure the accuracy of SketchVLMs and also how well they ground their markers on counting datasets (Sec. 4.1). We consider a marker

correct if it lies within the corresponding ground-truth bounding box, obtained via SAM-3 [7], allowing at most one marker per object.

Results ♦ sketchVLM exhibits strong consistency between counting accuracy (94.5) and numeric-marker location accuracy (95.9), whereas 🟢 sketchVLM achieves high counting accuracy (75.4) but substantially lower numeric-marker location accuracy (51.0), indicating that 🟢 sketchVLM often places markers incorrectly despite producing the correct count (Tabs. 2 and C3). 🤖 attains low counting accuracy (48.6) and numeric-marker location accuracy (59.9), indicating limited performance in both aspects (Tabs. 2 and C3). SketchVLM improves counting accuracy, yielding gains of (+1.5) points for ♦ and (+3.4) points for 🟢 (Tab. 2). The explicit numeric markers further enable direct visual verification of model outputs (Figs. D9 and 5).

5.4 SketchVLMs localize objects more accurately with pre-defined shape primitives than with free-form annotations 🟡✎

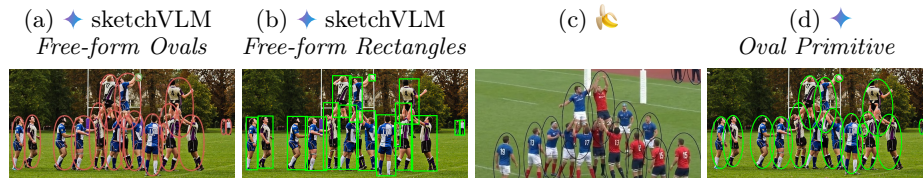


Fig. 6: When prompted to outline the classes “person” and “sports-ball”, (c) 🟡✎ replaces the original image with a newly generated one, whereas SketchVLM in (a) and (b) preserves the original image and draws shapes that accurately align with object boundaries and locations as compared to default (d) ♦.

A design choice we face is whether to have SketchVLMs generate all of their drawings through free-form strokes, or whether to allow them to output shape information (such as the x - y position of the center of the circle and the length of the radius) and then delegate the actual rendering of the drawing to the SVG conversion. Free-form strokes are more flexible but require point-by-point drawing and demand higher spatial precision. In contrast, pre-defined primitives for rectangles, ovals, etc. can be specified by a few parameters and can be rendered automatically. We compare both approaches in object localization in order to understand how it affects the output of annotations.

Experiment We compare SketchVLMs’ stroke-based annotations against the baseline models that directly output parameters for shape locations and shape size (Sec. 4.1). To evaluate oval annotations, we convert the rendered shape into its tight enclosing bounding box before computing metrics. Performance is measured using Average Precision (AP) at an IoU threshold of 0.5.

Results ♦ sketchVLM with stroke-based rectangles is effective for medium and large objects, but remains limited for small-object detection. Sketch-based

rectangles slightly improve performance on medium (+0.6) and large objects (+1.4), but significantly degrade small-object detection (-10.1), reducing overall AP50 from 63.1 to 58.8 (Figs. D10 and 6 and Tab. C4).

We observe that stroke-based outputs in SketchVLMs underperform the original model on small objects (Tab. C4). We examine detection statistics and prompt ablations and find that SketchVLMs match the original model in precision but exhibit lower recall (Tab. C5) and do not significantly change with variations in the drawing prompt (Tab. C6). Therefore, in order to get the accuracy of pre-defined shapes as well as the expressiveness of free-form strokes, we allow the model to output both simultaneously, such as in Fig. 1.

5.5 SketchVLM improves localization accuracy for 🌀 but not for ✨ when labeling parts of an object 🔗📷

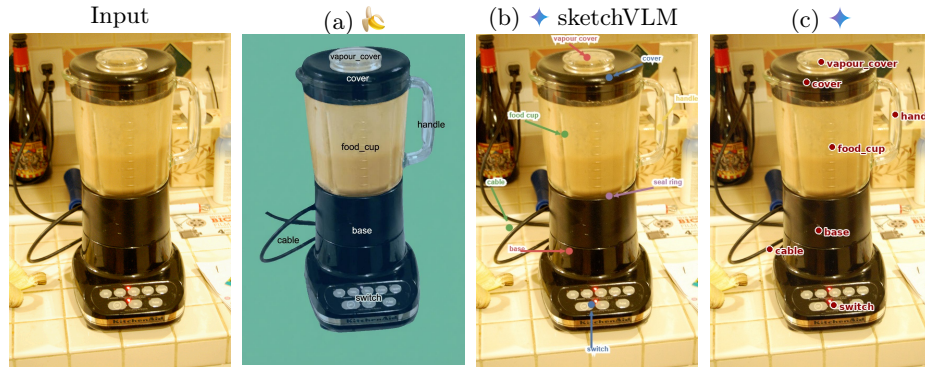


Fig. 7: Qualitative comparison on the part labeling task. (b) ✨ SketchVLM places each part label directly on its corresponding region while preserving the original image, producing more interpretable part annotations than (a) 🌀 or (c) ✨.

A useful feature of SketchVLMs is pointing at parts of an image and explaining them, for example, labeling engine components in a car maintenance guide (Fig. 1) or annotating regions in a screenshot (Fig. A1). We test SketchVLMs’ ability to produce text that is both semantically correct and spatially well-placed, a capability that underpins step-by-step instructions, explainable visual descriptions, and annotation tools [17, 28, 56].

Experiment In SketchVLM, the framework automatically renders the predicted text at the predicted location (with adaptive size/color for visibility), whereas the original VLM model outputs only the label and coordinates. For both SketchVLMs and the original VLMs, we prompt the models with a pre-defined set of valid part labels and require all predictions to be selected strictly from this set. For the original VLMs, we use a Python post-processing pipeline to overlay the predicted label onto the image at the predicted coordinates, where

Table 4: Labels placed by SketchVLMs land very close to the correct location. 🟢 sketchVLM is more accurate than the baseline at every tolerance level, and 🟠 sketchVLM matches it within a few pixels.

Dilation r (px)	🟠	🟠 sketchVLM	🟢	🟢 sketchVLM
0	64.1	60.3 (-3.8)	19.1	20.4 (+1.3)
3	69.0	66.5 (-2.5)	21.9	23.3 (+1.4)
5	70.8	69.3 (-1.5)	23.9	25.3 (+1.4)
7	71.8	71.2 (-0.6)	25.6	27.2 (+1.6)
10	73.1	73.5 (+0.4)	28.6	30.2 (+1.6)
15	75.1	78.3 (+3.2)	33.7	35.9 (+2.2)

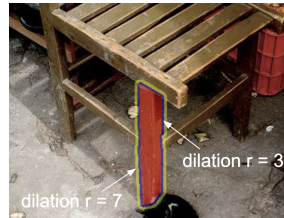


Fig. 8: From $r = 0$ (red) to modest dilation ($r = 7$, yellow), small boundary offsets become visually negligible.

text size and color are manually chosen for consistent visibility across examples. We follow [11] and define boundary dilation with radius r as expanding the ground-truth boundary by r pixels in all directions to allow tolerance in spatial matching.

Results SketchVLM improves part labeling for 🟢 (+1.3) but slightly underperforms 🟠 at strict boundary matching (-3.8) (Tab. 4). For 🟢, SketchVLM becomes increasingly robust under boundary dilation [11], achieving higher accuracy as tolerance increases, with remaining errors dominated by wrong-position mistakes (>79%; Tab. C7). For 🟠, the gap narrows from (-3.8) at $r = 0$ to (-0.6) at $r = 7$, reaching near parity under modest tolerance (Tab. 4). These errors largely correspond to small boundary offsets that are visually negligible (Figs. D11 7 and 8), where the original models produce more missing-label errors and SketchVLMs more position errors (Tab. C7). These findings indicate that SketchVLMs primarily introduce minor spatial shifts rather than semantic labeling failures, and remain competitive under reasonable boundary tolerance.

5.6 SketchVLMs outperform models fine-tuned directly on path-tracing tasks 🗺️

Maze Navigation evaluates spatial reasoning by requiring models to follow a sequence of directions to reach a goal while avoiding obstacles. Given that 🟡 and 🟢 are trained on a similar maze task, we expect them to perform well.

Experiment Given a set of directions, the model must sketch out the path while also determining if the path reaches the goal without crossing any walls.

Results Surprisingly, we find that 🟡 has an accuracy of 50.8% and 🟢 has an accuracy of 62.5%, both of which are near the random choice baseline of 50%. 🟡 + 🟠 performs much stronger with an accuracy of 93.3%; however, it sometimes alters the entire image and has other unusual outputs (Fig. D4). Both 🟠 sketchVLM and 🟢 sketchVLM perform well with accuracies of 98.0% and 92.8% respectively. The difference in performance between the default model baseline and the SketchVLM mode is minor (Tab. 2), but the annotations benefit user verification.

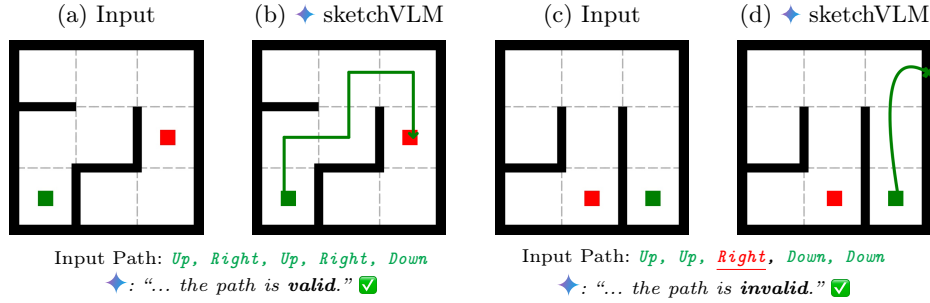


Fig. 9: Models are presented with a blank maze such as (a) or (c) and are asked to verify whether a proposed path from the green square to the red square is feasible through annotations (b) and (d). SketchVLMs correctly verify both valid and invalid paths by drawing the trajectory and marking where an invalid move occurs.

5.7 Fine-tuned sketching models fail to generalize to unseen physics understanding tasks 🏏

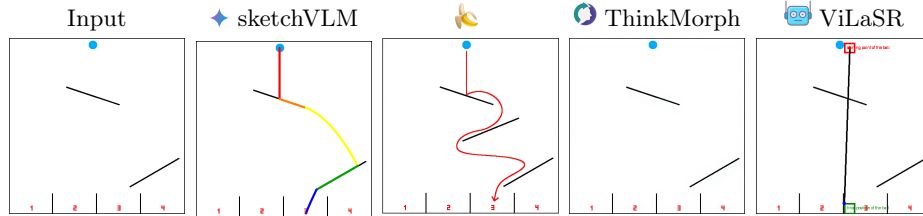


Fig. 10: SketchVLM generates the most accurate Ball Drop images compared to other baselines.

In addition to spatial reasoning, we evaluate whether SketchVLM can predict trajectories involving physical dynamics, such as a ball falling and rolling.

Experiment Given an image with a ball and platforms, the model must sketch the ball’s trajectory and output the container number it lands in.

Results We find that 🏏 and 🌀 perform poorly on VPCT with accuracies of 37% and 27%, which are near the random choice baseline of 33.3%. On our Ball Drop dataset, they have accuracies of 35.9% and 30.3%, only slightly above the random choice baseline of 25.0%, showing how these models fail to generalize to tasks that they are not trained on. 🏏 + ✦ often removes ledges in the image (Fig. D1) and performs significantly worse than the baseline ✦ accuracy (Tab. 2). In contrast, SketchVLMs output coherent annotations and have high accuracy, and ✦ sketchVLM reaches 96.0% accuracy on VPCT and 79.7% on Ball Drop (Fig. 10).

5.8 SketchVLM has higher annotation-text alignment than image-editing and fine-tuned models

Table 5: VLM-judged annotation-text alignment (higher is better) and annotation quality (1–5, higher is better). Models fine-tuned to generate annotations show lower alignment, while  sketchVLM has the highest alignment and annotation quality.

Model	Annotation-text Alignment				Annotation Quality (1-5)			
	VPCT	Ball Drop	Maze Navigation	Mean	VPCT	Ball Drop	Maze Navigation	Mean
 sketchVLM	99.0	99.0	88.4	95.5	1.83	1.74	3.20	2.26
 sketchVLM	100.0	98.5	84.2	94.2	3.12	4.28	3.69	3.70
 + 	93.0	41.4	80.3	71.6	1.56	2.56	3.68	2.60
	32.0	45.5	8.2	28.6	1.36	1.28	2.78	1.81
	54.0	38.4	48.1	46.8	1.62	2.11	1.17	1.63

Experiment To measure annotation-text alignment, we show each model’s annotations for VPCT, Ball Drop and Maze Navigation (without its text answer) to a VLM judge (Gemini-3-Flash-Preview [12]) and ask it to infer what the final answer should be from the annotations alone.

Results Despite being training-free, SketchVLMs achieve substantially higher annotation-text alignment than all baselines (Tab. 5).  sketchVLM and  sketchVLM reach mean alignment scores of 95.5% and 94.2%, respectively, compared to 71.6% for  + , 46.8% for , and 28.6% for . The low alignment of the fine-tuned models means their annotations frequently contradict their own text answers, making them unreliable as visual explanations. In contrast, SketchVLMs’ high alignment means that users can reliably look at the annotations to verify whether the model’s reasoning makes sense, which is the framework’s primary goal.

5.9 sketchVLM has significantly higher quality annotations than image-editing and fine-tuned models

Beyond faithfulness, an annotation must also be visually clear and logically coherent to be useful. A trajectory that clips through walls, a label that floats in empty space, or uninformative, overlapping annotations all undermine the user’s ability to interpret the model’s reasoning, even if the final text answer is correct (Fig. 11).

Experiment As described in Sec. 4.4, we measure drawing quality using a VLM-as-a-Judge approach with Gemini-3-Flash-Preview [12]. Each annotation is scored on a 1–5 rubric (Sec. E) in which the judge is asked to evaluate annotation plausibility and visual clarity.

Results Based on VLM-as-a-Judge ratings,  sketchVLM achieves the highest mean annotation quality score of 3.70, followed by  +  at 2.60,  sketchVLM

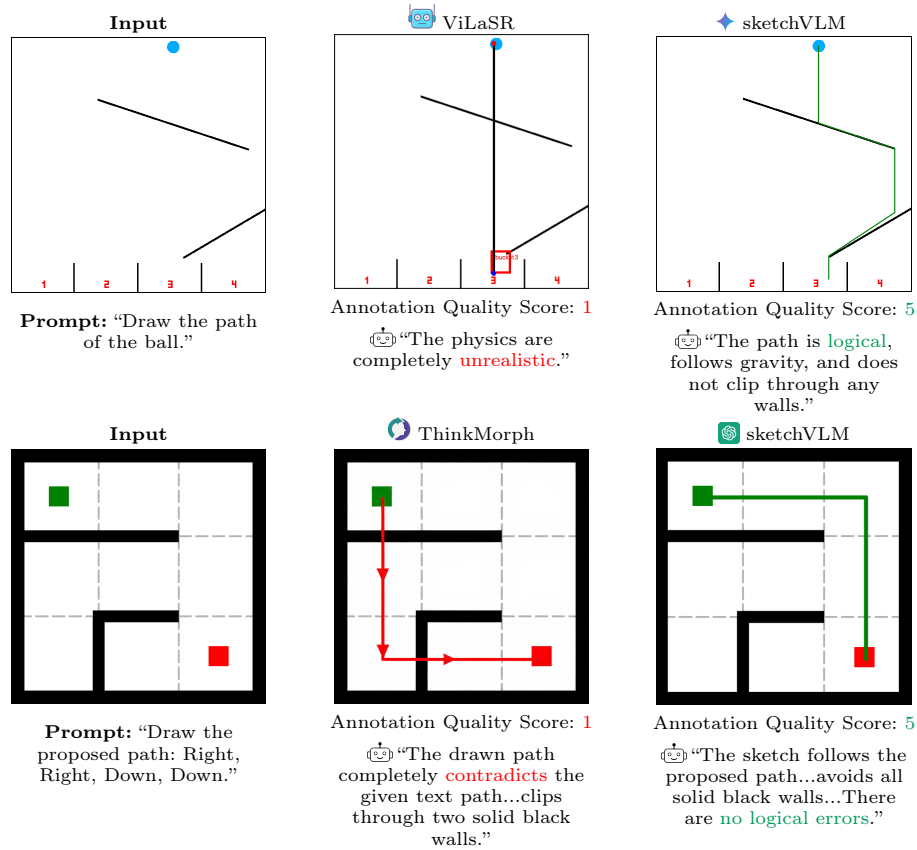


Fig. 11: Low-quality annotations from ThinkMorph and ViLaSR may still lead to the correct final answer, but contain logical errors that are harder for users to verify than the high-quality annotations from SketchVLMs.

at 2.26,  at 1.81, and  at 1.63 (Tab. [5](#)). Similarly, based on human ratings,  sketchVLM achieves the highest mean quality score of 4.14, followed by  sketchVLM at 3.70, Nano Banana Pro at 3.08,  at 1.74, and  at 1.24 (Tab. [E4](#)). Notably,  +  produces annotations that *appear* visually polished but sometimes contain subtle logical errors (e.g., trajectories passing through solid platforms) that even the VLM judge can miss (Sec. [E.1](#)). The fine-tuned models  and  score lowest, and manual inspection confirms that their annotations are often incoherent and difficult to interpret (Sec. [E.1](#)). These low quality scores also correlate with their low task accuracy in Tab. [2](#), suggesting that models unable to produce clear annotations also struggle to reason about the underlying tasks.

5.10 VLM judge ratings positively correlate with human judgments

We validate our annotation quality ratings from our VLM judge (Gemini-3-Flash-Preview [12] (Tab. F1)) against three human annotators on 2,250 annotations across Ball Drop, VPCT, and Maze Navigation using a quadratic Kappa score and Pearson Correlation. Agreement between humans and the VLM judge is moderate (quadratic Kappa of 0.51 ± 0.02 , Pearson correlation of 0.52 ± 0.01) compared to high inter-human agreement (quadratic Kappa of 0.84 ± 0.04 , Pearson correlation of 0.85 ± 0.04). Further details are in Sec. E. While the VLM judge is not as consistent as the human evaluators, its ratings are positively correlated with human judgments across all tasks and models. We therefore treat the VLM judge scores as a useful proxy metric for quick and cost-effective evaluation that should be taken into consideration with both task accuracy and annotation-text alignment.

5.11 Single-turn is as accurate as multi-turn but requires significantly fewer turns

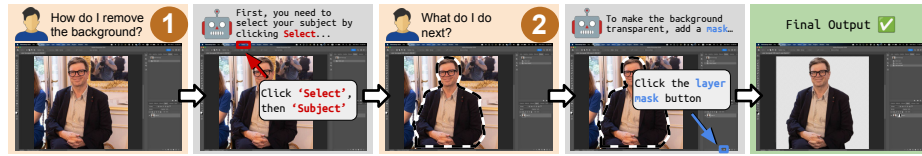


Fig. 12: Multi-turn example of SketchVLM guiding a user through how to remove an image’s background. At each turn, the model receives a screenshot then annotates the screenshot with labeled arrows and highlights UI elements to indicate the next step.

We want SketchVLMs to be able to visually guide users through tasks that require multiple turns, such as removing the background of a photo (Fig. 12) or setting up an EC2 instance on AWS (Fig. A1). We therefore evaluate how multi-turn generation affects model accuracy compared to single-turn, and identify the most effective configuration for multi-turn interaction.

Experiment We compare single-turn and multi-turn generation on VPCT, , , and . In multi-turn, the model receives all previously drawn strokes rendered onto the input image at each turn. We also test whether providing the text representation of rendered strokes across turns is necessary for maintaining annotation quality.

Results Single-turn achieves comparable or higher accuracy than multi-turn across all tasks (Tab. 3), while requiring about 5.92x fewer turns (Tab. C9). This demonstrates that SketchVLMs can produce high-quality annotations and accurate answers in a single pass. When we remove the text representation of prior strokes to force the model to rely solely on the rendered image, we observe notable degradation in annotation quality (Fig. D15). We therefore report all

multi-turn results with both the rendered image and the text history provided to the model.

6 Conclusion

We present SketchVLM, a training-free framework that prompts frontier VLMs to produce editable, non-destructive SVG annotations grounded on the input image. These visual explanations let users verify model reasoning at a glance, something that text-only responses and image-editing baselines fail to reliably provide. Our results show that this approach outperforms sketching models by +28.5 percentage points in accuracy (Tab. C1) and +48.3% in annotation quality (Tab. C2), while generalizing well to real-world tasks (Sec. A).

Limitations and Future Work SketchVLM works best with 🌀 and ✨ and can transfer to strong open-source VLMs like Kimi K2.5 [44] (Tab. C8 and Sec. D), but does not perform well on small VLMs that struggle with instruction following like Qwen2.5-VL-7B [3]. Additionally, enabling models to undo and edit strokes could be explored as a way to improve multi-turn performance.

7 Author contribution statement

Brandon Collins (BC), Logan Bolton (LB), and Hung Nguyen (HN) are major contributors who (a) created or curated benchmark datasets and (b) ran experiments.

- HN curated the Counting Objects, Drawing Shapes around Objects, and Part Labeling benchmarks from existing datasets, led their evaluation, and led the human-versus-VLM agreement study.
- BC created the Connect-the-Dots benchmark and led method development, inference and evaluation code, ablations, and evaluation on Connect-the-Dots and VPCT.
- LB created the Ball Drop and Maze Navigation benchmarks, led their evaluation, and ran the open-source model experiments as well as the annotation-quality and annotation-text alignment evaluations.

BC and LB led the writing of the manuscript while **all authors contributed to editing and reviewing**. BC, LB, and Anh Nguyen (AN) developed the demo. BC led development of the project website, with additional contributions from LB. BC and LB are technical team leads. AN supervised the project.

Acknowledgement

We thank Pooyan Rahmanzadehgervi at Auburn University for feedback and discussions of results. AN was supported by the NSF Grant No. 2145767, and donations from NaphCare Foundation & Adobe Research. LB was supported by the Auburn University URF program. HN was supported by the Auburn University AU PGRF program.